Functional Image File Format Release 1.4

- **Description**

Document ID: NM23918A-D
Publication date: October 2022
Language: English

MEGIN

MEGIN

Megin Oy

Keilasatama 5, FI-02150 Espoo, Finland

Tel: +358 9 756 2400
Fax: +358 9 756 24011
Web: www.megin.fi

MEGIN Customer Care
Tel +358 9 756 240 50
support@megin.fi

Printing History	Document ID	Fiff Version	Date
1st edition	NM20583A	1.1	April 2001
2nd edition	NM20583A-A	1.1	February 2003
3rd edition	NM20584A	1.2	January 2009
4th edition	NM23918A	1.3	April 2010
5th edition	NM23918A-A	1.3	September 2010
6th edition	NM23918A-B	1.3	March 2011
7th edition	NM23918A-C	1.4	August 2019
Change of office address	NM23918A-C1	1.4	February 2022
9th edition	NM23918A-D	1.4	October 2022

List of symbols

The following symbols may be used on the labels and in the manuals. Familiarize yourself with each symbol and its meaning before operating the MEG system.

Caution. Parts of the system are marked with this label when it is necessary to draw the attention of the user to avoid a potential hazard or to ensure safe, correct or improved operation or to avoid damage.

Refer to the instruction manual. Parts of the system are marked with this symbol when it is *mandatory* for the user to refer to instructions given in the manuals accompanying the system to ensure safe operation. In the manuals, it also calls attention to these instructions.

Consult instructions for use. Parts of the system are marked with this symbol when it is necessary for the user to refer to instructions given in the manuals accompanying the system. In the manuals, it also calls attention to these instructions. They are intended to ensure correct or improved operation and/or increased safety and to avoid damage.

Type BF (body floating) equipment symbol. The applied parts (parts in direct contact with the person being investigated with the system) and the type plate are marked with this symbol to indicate that they fulfill the leakage current requirements of the safety standard IEC 60601-1).

Alternating current (power line) symbol.

Protective ground (earth) terminal symbol. Used to identify terminals which are intended for connection to an external protective conductor for protection against electrical shock in case of a fault, or to the terminal of a protective ground (earth) electrode.

Static electricity symbol. The parts of the system marked with this symbol indicate the presence of components susceptible to static electricity and require the use of special static-electricity preventing techniques.

Non-ionizing radiation, RF transmitter. Marking on equipment or equipment parts that include RF transmitters or that intentionally apply RF electromagnetic energy.

Disposal instruction symbol. Separate collection of waste electrical and electronics equipment (WEEE) necessary (European Union directive 2012/19/EU on WEEE).

Name and registered address of the manufacturer, optionally combined with date of manufacture (year followed by month and day, if applicable).

Date of manufacture: year (four digits) followed by month and day (if applicable).

Model or type number.

Serial number.

Medical device.

Unique device identifier.

Contents

1

CHAPTER 1 Introduction

1.1 Background

Functional Image file format (FIFF) is a versatile format for representation of binary data. The basic structure of data allows to represent almost any collection of binary objects. Restrictions are posed mainly by the standardized set of “tags” and “types” that are used to give meanings to the data items, and to describe their encoding.

This manual contains information about the Functional Image File Format (FIFF) employed in the MEGIN software. It describes the syntax and semantics of the file format, and gives detailed description of the entities found when various types of data are stored using FIFF.

The discussion of the file format is structured as follows:

- Chapter 2 on page 7 describes the syntax of FIFF files.
- Chapter 3 describes the required structure of supported file types.
- Appendix A gives a directory of all the data types in the current version of the format.
- Appendix B gives a directory of all the enumerated data types in the current version of the format.
- Appendix C gives a directory of all the tags defined in the current version of the format.
- Appendix D gives a directory of all the information entities in the current version of the format.
-

1.2 Changes

This section lists the changes in this FIF format version compared to the version 1.3.

1.2.1 File Types

Structure definitions of supported file types have been updated. To conform with FIF format, any software should follow the definitions found in Chapter 3.

- Subject Information (Chapter 3.4.4): comment has been changed to subj_comment.
- Project Information (Chapter 3.4.5): comment has been changed to proj_comment.
- Device Information (Chapter 3.4.6) has been added.
- Helium Information (Chapter 3.4.7) has been added.
- Processing history (Chapter 3.4.8): MAXSHIELD_RECORD has been renamed IAS_RECORD.
- HPI chapter has been divided into three parts: HPI fitting results (Chapter 3.5.1), HPI measurement (Chapter 3.5.2) and Digitized points and strings (Chapter 3.5.3).
- Measurement Information (Chapter 3.5.5): experimenter is optional, HPI is replaced with HPI_RESULT, HPI_MEAS and ISOTRAK blocks, SUBJECT_INFO and PROJECT_INFO blocks are optional, CHANNEL_DECOUPLER, SSS_CAL_ADJUST, DEVICE_INFO and HELIUM_INFO blocks have been added, utc_offset has been added, bad_chs has been moved to the end.

1.2.2 Tags

The following new tags have been introduced in this FIF format:

- Common data tags (Section C.3): device_type (152), device_model (153), device_serial (154), device_site (155), he_level_raw (156), helium_level (157), orig_file_guid (158), utc_offset (159).
- Cross-talk (Section C.18): decoupler_matrix (800), ctm_open_amps (801), ctm_open_phase (802), ctm_clos_amps (803), ctm_clos_phase (804), ctm_clos_dote(805), ctm_open_dote(806), ctm_exci_freq (807).
- SSS operator matrices (Section C.19): sss_operator(290), sss_psinv (291), sss_ctc (292).

1.2.3 Types

The following type definitions in Appendix A have been changed:

- Channel info records (Section A.4.1): added optional ch_label.

1.2.4 Enumerated Types

The following *new entries to existing enumerated types* have been introduced:

- Value units (Section B.1): mol_m3, V_m2. Am_m2, Am_m3.
- Channel types (Section B.3): seeg, ecog, fnirs.
- Coil types (Section B.4): eeg_csd, fnirs_hbo, fnirs_hbr, fnirs_raw, fnirs_od, vv_planar_t4, vv_mag_t4, magnes_ref_mag, magnes_ref_gdia, magnes_ref_goff, ctf_ref_mag, ctf_ref_grad, ctf_ref_goff, kit_grad, kit_ref_mag, baby_grad, baby_mag_inner, baby_mag_outer, baby_mag_ref, artemis123_grad, artemis123_ref_mag, artemis123_ref_grad, quspin_zfopm_mag, quspin_zfopm_mag2, kriss_grad, compumedics_adult_grad, compumedics_pediatric_grad.
- Coordinate system definition (Section B.8): tufts_eeg, ctf_device, ctf_head, mri_voxel, ras, mni_tal, fs_tal_gtz, fs_tal_ltz, fs_tal.
- Cardinal points for brain (Section B.10): inion.
- SSS job options (Section B.12): sss_job_tproj, sss_job_xsss, sss_job_xsub, sss_job_xwav, sss_job_ncov, sss_job_scov.

The following *new enumerated types* have been introduced:

- Role (Section B.22)
- Hand (Section B.23)
- Map surface (Section B.24)
- Next (Section B.25)
- Sex (Section B.26)

1.2.5 Information Entities (Blocks)

The following new block types have been introduced:

- status_subsystem (block id = 123)
- device_info (124)
- helium_info (125)
- fiff_cov (355)
- sss_operator (506)
- ct_meas (507)

The following block types have been renamed:

- maxshield_raw_data is now ias_raw_data (119)
- maxshield_aspect is now ias_aspect (120)
- smartshield is now ias (510)

See Appendix D for details.

2

CHAPTER 2 FIFF file syntax

2.1 Introduction

A FIFF file contains a sequence of tag-data pairs. Each tag stores a particular piece of information about the data being stored. The tags form a linked list so that the logical sequence of tags may be different from the physical sequence found in the file. The last tag in the linked list has a 'nil' pointer assigned to it to indicate the end of the file. Except when specifically indicated this chapter considers the logical sequence set by the *next* members in the tag structure.

In addition to the main linked list, the file may contain additional linked lists consisting of tags which are used for maintenance purposes. Currently, two such lists are defined: the list containing the directory of the tags in the main list and a list of tags designated as free space in the file.

2.2 Tags

Each fiff tag has the members shown in Table 2.1. Each tagged object may contain zero or more bytes of data. The size of one tag-data pair in the file is thus 16 + *size_of_data* bytes. For more info about the data types used here, see Appendix A.

In the following we refer to the tags by the values of the *kind* member. All currently defined *kind*s are listed in Chapter C. For example, 'dir tag' refers to a tag whose kind member is 102, the value of defined by the 'id' column of Table C.3 on page 62. The value types are described in Appendix A and Appendix B.

NOTE: Because the next member of a FIFF tag is encoded with signed 32-bit integers, the FIFF file size should be limited to ***2 gigabytes***, otherwise the next member might overflow.

Name	Type	Size	Description
kind	int32_t	4	The kind of data stored in this tag.
type	int32_t	4	The data type of the contents.
size	int32_t	4	Number of bytes in the data member.
next	int32_t	4	Location of the next tag in the file. If this value is 0 , the next tag follows sequentially. If the value is -1 the list of tags ends here. Otherwise *next* indicates the byte position of the next tag in the file.
data	byte_t*	variable	*size* bytes of data contained in the tag.

Table 2.1 A fiff tag.

2.3 Information entities

Individual tags are grouped into meaningful information entities by the block_start and block_end tags. They act like the braces in C programs first by defining a meaningful grouping. They also allow a definition of 'scope'. The latter aspect is further discussed later when specific FIFF file contents are described in Chapter 3.

The data in the block_start and block_end identifies the kind of block enclosed. The data member would not be absolutely necessary in the block_end tag but is included for symmetry and to allow for some error checking in the file. The allowed data values are listed in Appendix D.

2.4 Tag directory

The purpose of the tag directory is to facilitate searching of information from a FIFF file. Without the directory, the tag list would have to be read from the beginning until the desired data are located. The data of dir tag, which dir_pointer tag points to, is an array of *dir_entry_t* structures (see Section A.4.3). Programs can read the directory into memory for searching the contents, and then access the data through pointers in the directory.

The tag directory is not mandatory. Programs using FIFF files should be able to manage files with or without the directory.

3

CHAPTER 3 File types

3.1 Introduction

This section describes the supported file types and the required content of each file.

3.2 Conventions

As described in section Chapter 2 the content is structured into information entities (blocks) that contain tags. Both tags and blocks can have a multiplicity ≥ 0. Some tags and blocks are mandatory, *i.e.*, required to make up a complete file the specific data type, and some tags and blocks are optional, *i.e.*, the file should be usable without them, and should be supported by the compatible programs.

It is assumed that most of the tags in the file can be ordered in any order inside a block, and that most of blocks can be ordered in any order inside the file. The programs that read files should not assume anything from the ordering of the tags, besides the exceptions listed below:

- The file must start with the tags defined in section 3.4.2, ordered as specified in the section.
- The data tags (data_buffer, data_skip, and data_skip_samples) in raw and continuous data files must be ordered sequentially starting from the first data tag in time. The channels in data tags should be ordered in scanning order (scanNo member of *ch_info_t* struct). See Section 3.5.6 for details on raw and continuous data file structure.
- The evoked file data tags (epochs) encoded with *old_pack_t* (see Section A.3.1) must be ordered in scanning order (scanNo member of *ch_info_t* struct). The channels in evoked file data tags encoded as matrices should be ordered in scanning order (scanNo member of *ch_info_t* struct). See Section 3.5.7.2 for details on evoked data file structure.

To provide compability with files in future FIFF versions, the programs reading FIF files should disregard any tags unknown to them.

3.3 Structure syntax

This section describes the syntax used below for describing the structures.

3.3.1 Tags

Tags are identified by names defined in Appendix C written in lowercase characters. The data type of each tag is omitted. See Appendix C for the data type and description of each tag. The basic and structured data types are described in Appendix A and the enumerated types in Appendix B.

3.3.2 Blocks

A block is described as a C-like block inside curly brackets. The type of the block is given as a variable name found in Appendix D before the starting bracket. The tags inside the block are indented once compared to the level of the block name.

An example:

```
block {
    tag1
    tag2
}
```

In general, no requirement is set for the ordering of tags inside a block, or blocks inside a block.

3.3.3 Segment definitions

Common segments are identified by names written in uppercase characters:

```
SEGMENT =
    tag
    block {
         tag
    }
```

The segments can be used inside other definitions. The segment name can be replaced by the text on the right hand side of the segment definition:

```
tag
SEGMENT
block {
    tag
}
```

3.3.4 Multiplicative and optional statements

If the tag/block appears only once, and is required, it is written as it is:

```
block {
   tag
}
```

If the tag/block is required to appear n times, a number is used inside square brackets:

```
block[n] {
   tag[n]
}
```

If the tag/block has to appear at least once, but can appear any number of times (one or more), plus sign is used:

```
block+ {
   tag
}
```

If the tag/block is optional, a question mark is used:

```
block? {
   tag?
}
```

Also multiplicative statements can be optional. Star sign is used if the element can appear any number of times, but it is possible to leave it out (zero or more):

```
block* {
   tag*
}
```

The question mark is placed after the square brackets to signify that the element can be omitted, but if it appears, it has to be repeated n times:

```
block[n]? {
   tag[n]?
}
```

If there are two interchangeable options for tag/block it is denoted by '|' between the options:

```
block1 | block2 {
   tag1 | tag2
}
```

3.4 Common structures for different file types

This section describes information that is common to all the different file types.

3.4.1 Common to blocks

It is preferred that a block is tagged with an identifier that allows it to be uniquely identified (block_id tag) and referenced. In addition, if the block has been generated from some other block, possibly in some other file, the identifier of the block the new block was derived from should be stored in the modified file for reference (parent_block_id tag).

```
COMMON_TO_BLOCKS
     block {
           block_id?
           parent_block_id?
     }
```

3.4.2 File start

The file has to start always with the same sequence of tags:

```
FILE_START =
     file_id
     dir_pointer
     free_list?
     parent_file_id?
```

3.4.3 File end

It is customary that the file ends to a nop tag. The nop tag can be used also in other places to indicate a place where additional information might be stored.

3.4.4 Subject information

This entity contains information about the subject.

```
SUBJECT_INFO =
     subject {
          subj_id
          subj_his_id?
          subj_last_name
```

```
        subj_first_name
        subj_middle_name?
        subj_birth_day?
        subj_sex?
        subj_hand?
        subj_weight?
        subj_height?
        subj_comment?
    }
```

3.4.5 Project information

This entity contains information about the project under which the file has been produced.

```
PROJECT_INFO =
    proj_id
    proj_name
    proj_comment?
```

3.4.6 Device information

This entity contains information about the MEG device on which the file has been produced.

```
DEVICE_INFO =
    device_info {
        device_type
        device_model?
        device_serial?
        device_site?
    }
```

3.4.7 Helium information

This entity contains information about the latest helium level measurement.

```
HELIUM_INFO =
    helium_info {
        he_level_raw?
        helium_level?
        gantry_angle?
        meas_date
    }
```

3.4.8 Processing history

Information on any processing performed on a file should be stored inside a processing_history block. The information is stored inside individual processing record blocks.

```
PROCESSING_HISTORY =
     processing_history {
          PROCESSING_RECORD+
     }
```

The processing record has a couple required tags, and optionally can contain specific information on the processing performed (denoted by PROCESSING_INFO+, which can be either tags or blocks).

```
PROCESSING_RECORD =
     processing_record {
          meas_date
          experimenter
          creator?
          PROCESSING_INFO+
     }
```

For MEG signal data two processing record blocks are defined in the format: SSS_RECORD and IAS_RECORD (see the following sections). User defined processing record blocks can also be used in all the supported files.

3.4.8.1 SSS processing record

This section describes the SSS processing record block.

```
SSS_INFO =
     sss_info {
          sss_job
          sss_frame
          sss_origin
          sss_ord_in
          sss_ord_out
          sss_nmag
          sss_components
          sss_nfree
     }

CHANNEL_DECOUPLER =
     channel_decoupler {
          meas_date
          creator
          decoupler_matrix
          sparse_ch_name_list
     }
```

```
SSS_CAL_ADJUST =
     sss_cal_adjust {
          sss_cal_chans
          sss_cal_corrs
     }

SSS_RECORD =
     processing_record {
          meas_date
          experimenter
          creator
          SSS_INFO
          CHANNEL_DECOUPLER
          SSS_CAL_ADJUST
     }
```

3.4.8.2 IAS™ processing record

This section describes the IAS processing record.

```
IAS_RECORD =
     processing_record {
          meas_date
          experimenter
          ias {
          }
     }
```

3.5 MEG signal data

This section describes the structures used for encoding raw, evoked or processed MEG signal data.

3.5.1 Head position (HPI) fitting results

This entity contains information about the HPI fitting.

```
HPI_RESULT =
    hpi_result {
        dig_point*
        hpi_digitization_order
        hpi_coils_used
        hpi_coil_moments
        hpi_fit_goodness
        hpi_fit_good_limit
        hpi_fit_dist_limit
        hpi_fit_accept
        coord_trans
    }
```

3.5.2 Head position (HPI) measurement

This entity contains information about the HPI measurement.

```
HPI_MEAS =
    hpi_meas {
        creator
        sfreq
        nchan
        nave
        hpi_ncoil
        first_sample
        last_sample
        hpi_coil+ {
            hpi_coil_no
            epoch+
            hpi_slopes
            hpi_corr_coeff
            hpi_coil_freq?
        }
    }
```

3.5.3 Digitized points and strings

This entity contains the digitized points and strings.

```
ISOTRAK =
     isotrak {
          meas_date?
          dig_point+
          dig_string*
     }
```

3.5.4 SSP

This section describes signal space projection (SSP) information.

```
SSP =
     ssp {
          nchan?
          proj_item+ {
               description
               proj_item_kind
               nchan?
               proj_item_ch_name_list
               proj_item_time
               proj_item_nvec
               proj_item_vectors
          }
     }
```

3.5.5 Measurement information

This section describes the whole measurement information segment. See above for definitions of HPI_RESULT, HPI_MEAS, ISOTRAK, SUBJECT_INFO, PROJECT_INFO, DEVICE_INFO, HELIUM_INFO, and SSP segments.

See Chapter 3.4.8.1 for definitions of CHANNEL_DECOUPLER and SSS_CAL_ADJUST which, if present in a raw data file, are used by MaxFilter™ and subsequently moved to processing_history block.

```
MEAS_INFO =
     meas_info {
          experimenter?
          description?
          events? {
               event_channels
               event_list
          }
```

```
        HPI_RESULT?
        HPI_MEAS?
        ISOTRAK?
        dacq_pars? {
            dacq_pars
            dacq_stim?
        }
        SUBJECT_INFO?
        PROJECT_INFO?
        SSP?
        CHANNEL_DECOUPLER?
        SSS_CAL_ADJUST?
        DEVICE_INFO?
        HELIUM_INFO?
        meas_date
        utc_offset?
        nchan
        sfreq
        lowpass
        highpass
        line_freq?
        data_pack?
        gantry_angle?
        ch_info[nchan]
        hpi_subsystem? {
            hpi_ncoil
            event_channel
            hpi_coil+ {
                  event_bits
            }
        }
        bad_chs?
    }
```

3.5.6 Raw data files

Raw data is data that comes directly from the data acquisition system.

The raw data block starts with an optional first_samp tag. The tag indicates the starting time of the data. To get the time in seconds, divide the first_samp by sampling frequency (sfreq tag). If first_samp tag is not available, it is assumed to be zero. The time is relative to the measurement time defined by the meas_date tag inside the meas_info block.

After the first_samp, the raw data block can contain any number of data buffers and data skips. Data skip is a time segment where no data has been measured. There are two ways of encoding a data skip. Currently only only data_skip tag is supported, but any data reading routines should be designed to handle also data_skip_samples tag. The difference is that data_skip gives the length of the skip in number of data buffers whereas

data_skip_samples gives the length in samples. When using data_skip you have to deduce the size of the data skip in samples (see below how to compute the buffer size) from the first data buffer in the file, and multiply the number of data buffers indicated by data_skip with this number to get the length of the skip.

The data buffer contains data sample vector by sample vector. A sample vector contains one simultaneous sample from each channel. Buffer size should be constant inside a raw data block. The only exception is that the last buffer can be smaller than the other buffers. The buffer size in samples can be computed from the size of tags by dividing it with the number of channels and the size of the data type used to store the data.

To get the data in real values described by unit and unit_mul in ch_info_t, you have to multiply the data values in file by each channels's ch_info_t's cal and range parameters (see Table A.3).

The raw data is usually stored into a single file. Due to the 2 gigabyte limit in the file size (see Section 2.2 for details), the raw data files are split into multiple files whenever this limit might be exceeded. The files can be split also into smaller segments. See Section 3.5.6.1 for information on how to encode information in single raw data file. Section 3.5.6.2 describes the referencing mechanism used when splitting the data into multiple files.

3.5.6.1 Raw data in a single file

The raw data is stored inside a raw_data block. Any off-line processing software should store its processed output inside processed_data block as described in Section 3.5.7. However, some programs do not use the processed_data block, and might store their output inside raw_data block.

```
FILE_START
measurement {
     MEAS_INFO
     PROCESSING_HISTORY?
     raw_data {
          first_samp?
          (data_buffer | data_skip | data_skip_samp)*
     }
}
nop
```

3.5.6.2 Raw data in multiple files

When data is split into multiple files the following reference block is used to store references between the files:

```
FILE_REFERENCE =
     ref {
          ref_role
          ref_file_id
          ref_file_num
     }
```

where ref_role defines whether this reference is to the file before (prev_file) or after (next_file) this file in the multiple sequence file sequence (see Appendix B.22), ref_file_id is the identifier of the referenced file, and ref_file_num is the running number of the referenced file in the multiple file sequence.

The FILE_REFERENCE segment is stored inside raw_data block. The FILE_REFERENCE in the referencing file can be replaced by the data tags (data_buffer, data_skip or data_skip_samp) of the referenced file. In other words, the first file in the sequence would be code as:

```
FILE_START
measurement {
     MEAS_INFO
     raw_data {
          first_samp?
          (data_buffer | data_skip | data_skip_samp)*
          FILE_REFERENCE
     }
}
nop
```

i.e. a FILE_REFERENCE to the next file in the sequence is stored after the data tags. If the data is split into 3 or more files, the intermediate files have FILE_REFERENCEs to both the previous and the next files in the sequence:

```
FILE_START
measurement {
     MEAS_INFO
     raw_data {
          FILE_REFERENCE
          first_samp?
          (data_buffer | data_skip | data_skip_samp)*
          FILE_REFERENCE
     }
}
nop
```

The last file in the sequence has reference only to the previous file, and hence ends the sequence:

```
FILE_START
measurement {
     MEAS_INFO
```

```
    raw_data {
        FILE_REFERENCE
        first_samp?
        (data_buffer | data_skip | data_skip_samp)*
    }
}
nop
```

3.5.6.3 Raw IAS™ data

When data has been measured with a system where Internal Active Shielding (IAS™) is turned on, the data is stored inside an ias_raw_data block. Additionally, an IAS_RECORD segment is stored inside a processing_history block.

This section describes only IAS™ data stored into a single file. IAS™ data split into multiple files is encoded the same as the normal raw data (see Section 3.5.6.2).

Warning: The data inside ias_raw_data block must be processed with MaxFilter™ before using it for analysis to produce reliable results. The raw IAS™ data may be distorted.

```
FILE_START
measurement {
    MEAS_INFO
    processing_history {
        IAS_RECORD
    }
    ias_raw_data {
        first_samp?
        (data_buffer | data_skip | data_skip_samp)*
    }
}
nop
```

3.5.7 Processed data files

There are two types of processed MEG signal data: continuous and evoked. The continuous data is used for storing long segments of data, such as result of filtering the whole raw data file. Evoked data is used for storing smaller segments of data, of the order of seconds. One special application for evoked data is averaging. The averaged data categories can be stored as evoked blocks in an evoked data file.

3.5.7.1 Continuous data

The continous data should be stored inside a continuous_data block. The continuous_data block is stored inside a processed_data block. Note that there are programs that write processed data inside a raw_data block (as described in Section 3.5.6).

```
FILE_START
measurement {
     MEAS_INFO
     PROCESSING_HISTORY
     processed_data {
          continuous_data {
               first_samp?
               (data_buffer| data_skip |
               data_skip_samp)*
          }
     }
}
nop
```

This section describes only continuous data stored into a single file. Continuous data split into multiple files is encoded the same as the raw data in multiple files (see Section 3.5.6.2).

3.5.7.2 Evoked data

The evoked data is stored in continuous epochs inside one or more evoked blocks. Data in each evoked block can differ in size from others.

```
FILE_START
measurement {
     MEAS_INFO
     PROCESSING_HISTORY?
     processed_data {
          evoked+ {
                  ref_event
                  first_sample
                  last_sample
                  event_comment?
                  comment?
                  aspect {
                     aspect_kind
                     nave?
                     epoch+
                  }
          }
     }
}
nop
```

3.5.7.3 Evoked IAS™ data

When data has been measured with a system where Internal Active Shielding (IAS™) is turned on, the on-line and off-line average of the data is stored inside an ias_aspect block. Additionally, an IAS_RECORD segment is stored inside a processing_history block.

Warning: The data inside ias_aspect block must be processed with Max-Filter™ before using it for analysis to produce reliable results. The raw IAS™ data may be distorted.

```
FILE_START
measurement {
     MEAS_INFO
     processing_history {
          IAS_RECORD
     }
     processed_data {
          evoked+ {
                  ref_event
                  first_sample
                  last_sample
                  event_comment?
                  comment?
                  ias_aspect {
                      aspect_kind
                      nave?
                      epoch+
                  }
          }
     }
}
nop
```

3.6 Volume image data

3.6.1 Pixel data included in the FIFF file

This section describes how pixel data is included in the FIFF file.

```
PIXEL_DATA_INCLUDED =
     mri_pixel_data
     mri_pixel_type?

     mri_orig_source_path?
     mri_orig_source_format?
```

3.6.2 Pixel data referenced from an external file

This section describes how pixel data is referenced from other files.

```
PIXEL_DATA_REF =
     ref_path
     mri_source_format
     mri_pixel_type
     mri_pixel_data_offset
```

3.6.3 Volume data

This section describes the volumetric (or structural) image data file.

```
FILE_START
structural_data {
     SUBJECT_INFO?
     PROCESSING_HISTORY?
     volume_data+ {
          coord_trans?
          dig_point[3]?
          volume_slice+ {
               mri_width
               mri_width_m
               mri_height
               mri_height_m
               coord_trans
               PIXEL_DATA_INCLUDED | PIXEL_DATA_REF
               mri_pixel_scale
          }
     }
}
nop
```

3.7 Image capture data (scenes)

This section describes screen capture data file.

```
FILE_START
scenery {
     SUBJECT_INFO?
     PROCESSING_HISTORY?
     coord_trans
     mri_scene_aim
     scene+ {
          mri_width
          mri_width_m
          mri_height
          mri_height_m
          coord_trans
          block_name
          mri_pixel_data
          mri_source_format
          mri_pixel_scale
     }
}
nop
```

3.8 Triangulated mesh data

This section describes triangulated mesh data files used for boundary element modelling.

```
FILE_START
bem {
     SUBJECT_INFO?
     PROCESSING_HISTORY?
     bem_coord_frame
     coord_trans
     bem_surf+ {
          bem_surf_id
          bem_sigma
          bem_surf_nnode
          bem_surface_ntri
          bem_surface_nodes
          bem_surface_normals?
          bem_surface_triangles
     }
     bem_approx?
     bem_pot_solution?
}
nop
```

A

APPENDIX A Data type directory

This chapter describes what data types are available and how they are encoded in the FIFF files. Section A.1 describes byte ordering used in encoding the data. Section A.2 lists supported data type identifiers and their values. Section A.3 describes the basic data types. The basic data types can be used as building blocks of different structures (described in Section A.4) or multidimensional data (e.g. vectors and matrix, described in Section A.5.

A.1 Byte ordering

FIFF was originally designed for the 'big-endian' (Sun, HP) byte ordering in integers. It was also assumed that the floating point numbers are presented using the IEEE standard. The data in files are always stored in this ordering and floating-point representation irrespective of the computer platform.

The FIFF library described in a separate manual takes care of the necessary conversions of the byte order and floating point representation (e.g. in Intel i686 architecture) so that this choice will be transparent to the user, if the FIFF library is used to access the files.

A.2 Data type identifiers

The allowed values of the *type* member of a tag are enumerated in Table A.1. The tag may contain a single value of the indicated type or a linear array thereof. The program reading the file should deduce the existence of a vector (array) by checking the *size* member and comparing it to the size of the data type corresponding to the value of the *type* member.

The type members 12 least significant bits are devoted for storing the data type identifier. The rest of the bits are used for storing information on whether the data is in different matrix formats. See Section A.5.2.1 for details on how to encode matrices in FIFF.

The data types listed in Data type column of the Table A.1 are described in sections A.3-A.5.2.2. In addition, the tags maybe described by enumerated types. These enumerated types are listed in Appendix B.

Data type *stream_segment_t* is used internally by the acquisition system, and is not explained in this document..

Name	ID	Data type	Base size	Reference
void	0	void_t	1	Section A.3
byte	1	byte_t	1	Section A.3
int16	2	int16_t	2	Section A.3
int32	3	int32_t	4	Section A.3
float	4	float_t	4	Section A.3
double	5	double_t	8	Section A.3
julian	6	julian_t	8	Section A.3
uint16	7	uint16_t	2	Section A.3
uint32	8	uint32_t	4	Section A.3
uint64	9	uint64_t	8	Section A.3
string (iso-8859-1)	10	byte_t	1	Section A.3
ascii	10	byte_t	1	Section A.3
int64	11	int64_t	8	Section A.3
dau_pack13	13	dau_pack13_t	2	Obsolete
dau_pack14	14	dau_pack14_t	2	Obsolete
dau_pack16	16	dau_pack16_t	2	Section A.3
complex_float	20	complex_float_t	8	Section A.3.2
complex_double	21	complex_double_t	16	Section A.3.2
old_pack	23	old_pack_t	variable	Section A.3.1
ch_info_struct	30	ch_info_t	96 or 112	Section A.4.1
id_struct	31	id_t	20	Section A.4.2
dir_entry_struct	32	dir_entry_t	16	Section A.4.3
dig_point_struct	33	dig_point_t	20	Section A.4.4
ch_pos_struct	34	ch_pos_t	52	Section A.4.5
coord_trans_struct	35	coord_trans_t	80	Section A.4.6

Table A.1 Values allowed for the *type* member of a FIFF tag.

Name	ID	Data type	Base size	Reference
dig_string_struct	36	dig_string_t	variable	Section A.4.4
stream_segment_struct	37	stream_segment_t	internal	internal

Table A.1 Values allowed for the *type* member of a FIFF tag.

A.3 Basic data types

The lowest level of FIFF file structure consists of basic data types that are used in saving data. All data can be expressed in terms of these types, so they are the foundation of the format. These data types match with the basic data types of C language, so that they can be easily expressed in C or C++. Note, however, the definitions of these data types are synonymous to the data types actually found in a FIFF file only in big-endian byte ordering systems using the IEEE floating point representation. Other platforms must use proper conversion routines when writing or reading data from a file.

The basic data types are listed in Table A.2. These data type names are used when presenting the contents of FIFF files.

Data type	Size bytes	Description
void_t	0	No data
byte_t	1	1-byte unsigned char
int16_t	2	Signed 2-byte integer
uint16_t	2	Unsigned 2-byte integer
int32_t	4	Signed 4-byte integer
uint32_t	4	Unsigned 4-byte integer
int64_t	8	Signed 8-byte integer
uint_64_t	8	Unsigned 8-byte integer
float_t	4	Single-precision 4-byte floating point number (IEEE)
double_t	8	Double-precision 8-byte floating point number (IEEE)
dau_pack16	2	Essentially the same as int16_t

Table A.2 Basic data types.

A.3.1 Encoding of responses with old_pack_t

If the *type* member of a tag is set to *old_pack*, a special encoding scheme is used. This encoding is only used for evoked responses. If the number of samples in the data is *nsamp*, the packed data contains two *float_t* floating point numbers followed by *nsamp* 2-byte signed integers (*int16_t*). The size of the tag data is thus $2 \times 4 + nsamp \times 2$ bytes. The two floats give the offset (b) and and scale (a) for the data so that the original floating point data f_k are packed to integers s_k as

$$s_k = (f_k - b)/a$$

Unpacking is accomplished simply by

$$f_k = a \times s_k + b \ .$$

A.3.2 Complex numbers

There are two different complex number formats defined: *complex_float_t* and *complex_double_t*. The former is encoded with two consequent *float_t* numbers and the latter with two *double_t* numbers.

A.3.3 Julian date

Data type *julian_t* encodes a date in julian calender. The value is encoded using int32_t.

A.4 Structures

In addition to the basic data types, some objects in the FIFF files have data type which is a structure. A structure consists of a set of objects having basic data type. Structures are packed using alignment at four byte boundaries (relative to the start of the structure).

The tables of this section list the data structures used in the FIFF files. The members of the structures are listed in the ‘Member’ column. The ‘Size’ column gives the size of the members. The final column gives the description of the members.

A.4.1 Channel info records

The *ch_info_t* structure (Table A.3) contains information about one channel.

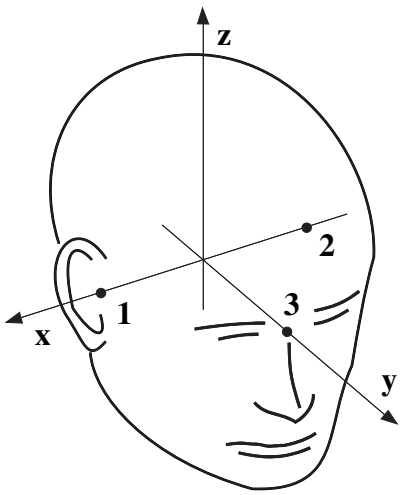

Figure A.1 The head coordinate system.

The positions of MEG channels are given in device coordinates while the position of the EEG channels are obtained in the head digitization procedure and are thus given in head coordinates. Both coordinate systems are right-handed Cartesian coordinate systems. The direction of x axis is from left to right, that of y axis to the front, and the z axis thus points up.

The x axis of the head coordinate system passes through the two periauricular or preauricular points digitized before acquiring the data with positive direction to the right. The y axis passes through the nasion and is normal to the x axis. The z axis points up according to the right-hand rule and is normal to the xy plane.

The origin of the device coordinate system is located at the center of the posterior spherical section of the helmet with x axis going from left to right and y axis pointing front. The z axis is, again normal to the xy plane with positive direction up.

Member	Type	Size	Description
scanNo	int32_t	4	Position of this channel in scanning order.
logNo	int32_t	4	Logical channel number. These must be unique within channels of the same kind.
kind	int32_t	4	Kind of the channel described (MEG, EEG, EOG, etc.)
range	float_t	4	The raw data values must be multiplied by this to get into volts at the electronics output.

Table A.3 The channel information data structure *ch_info_t*.

Member	Type	Size	Description
cal	float_t	4	Calibration of the channel. If the raw data values are multiplied by *range*cal*, the result is in units given by *unit* and *unit_mul*.
coil_type	enum(coil)	4	Kind of MEG coil or kind of EEG channel.
r0[3]	float_t	12	Coil coordinate system origin. For EEG electrodes, this is the location of the electrode.
ex[3]	float_t	12	Coil coordinate system unit vector $\boldsymbol{e}_x$. For EEG electrodes, this specifies the location of a reference electrode. Set to (0,0,0) for no reference.
ey[3]	float_t	12	Coil coordinate system unit vector $\boldsymbol{e}_y$. This is ignored for EEG electrodes.
ez[3]	float_t	12	Coil coordinate system unit vector $\boldsymbol{e}_z$. This is ignored for EEG electrodes.
unit	enum(unit)	4	The real-world unit-of measure.
unit_mul	enum(unitm)	4	The unit multiplier. The result given by *range*cal*data* is in units *unit*10^unit_mul*.
ch_name [16]	byte_t	16	Device name of the channel terminated with 0 (e.g. “MEG0111” or “EEG001”)
ch_label [16]	byte_t	16	Optional anatomical name of the channel terminated with 0 (e.g. “LT371” or “Fp1”)
		96	Total size without optional members.
		112	Total size with optional ch_label.

Table A.3 The channel information data structure *ch_info_t*.

A.4.2 Globally unique identifiers

A special object *id_t* (Table A.4) is used to hold a globally unique identifier, which can be used for various purposes in the FIFF file. The internal structure of the identifier is defined here, but usage of the information of the fields in the identifier is deprecated, since their contents may be redefined to support some future platforms. For example the creation time fields can contain something else than true time information in objects created on a platform that does not support a real time clock. There is one exception, the file_id tag at the start of the file should contain the version information of the FIFF format used by the file.

Member	Type	Size	Description
version	int32_t	4	The FIFF format version used. The higher and lower 16 bits are used to code the major and minor version numbers respectively.
machid[2]	int32_t	8	Unique machine id. Usually contains the hardware address of the primary LAN card.
time_sec	int32_t	4	Time (seconds from epoch) of the time when the id was created.
time_usec	int32_t	4	Time (fraction in micro seconds) of the creation time.
		20	Total size

Table A.4 The identifier structure *id_t*.

A.4.3 Directory entries

The *dir_entry_t* structure (Table A.5) holds one entry in a directory of tags used to allow quick access to the information in a FIFF file (see Section 2.4).

Member	Type	Size	Description
kind	int32_t	4	Kind of the tag this entry refers to.
type	int32_t	4	Data type of the object.
size	int32_t	4	Size of the object data.
pos	int32_t	4	Position of the tag in the file.
		16	Total size

Table A.5 The directory entry structure *dir_entry_t*.

A.4.4 3D coordinates

The following two data structures are related to the head digitization. The *dig_point_t* structure (Table A.6) represents one point, *e.g.*, the location of a cardinal landmark or that of an EEG electrode. The *dig_string_t* structure (Table A.7) is used to represent a set of points acquired in the continuous digitization mode.

members	member type	Size	Description
kind	int32_t	4	What kind of point is this.
ident	int32_t	4	Running number for this point.
r[3]	float_t	12	The location of the point in meters.
		20	Total size

Table A.6 The digitization point structure *dig_point_t*.

members	member type	Size	Description
kind	int32_t	4	What kind of points these are.
ident	int32_t	4	Running number for this point set.
np	int32_t	4	Number of points in this set.
**rr	float_t	var	An array of pointers to locations. Each pointer points to an array of three floats.
		var	Total size

Table A.7 The digitization string structure *dig_string_t*.

A.4.5 Channel position information

The *ch_pos_t* structure is used for representing coil position information. The members are described in

Member	Type	Size	Description
coil_type	int32_t	4	What kind of coil
r0[3]	float_t	12	Coil coordinate system origin
ex[3]	float_t	12	Coil coordinate system x-axis unit vector
ey[3]	float_t	12	Coil coordinate system y-axis unit vector
ez[3]	float_t	12	Coil coordinate system z-axis unit vector
		52	Total size

Table A.8 The channel position information structure *ch_pos_t*.

A.4.6 Coordinate transformations

The *coord_trans_t* structure employed to represent coordinate frames is listed in Table A.9. The *from* and *to* members select the coordinate system to which the coordinate transformation applies. The *from* coordinate system is the coordinate system where the input for the forward transformation is assumed to be in, i.e. the source. The *to* coordinate system is the coordinate system where the input is transformed to by the forward transformation, i.e. the destination. Available coordinate systems are listed in Table B.8.

Member	Type	Size	Description
from	int32_t	4	The source coordinate frame.
to	int32_t	4	The destination coordinate frame.
rot[3][3]	float_t	36	Rotation matrix between the coordinate frames. This is denoted by $\boldsymbol{R}$ in the text. Because $\boldsymbol{R}$ describes a rotation, we have $\boldsymbol{R}^T\boldsymbol{R} = \boldsymbol{I}$.
move[3]	float_t	12	Location of the source coordinate system origin in destination coordinates. This column vector is denoted by $\boldsymbol{r}_{0s}$ in the text.
invrot[3][3]	float_t	36	Inverse rotation matrix between the coordinate frames. This is denoted by $\boldsymbol{R}^T$ in the text.
invmove[3]	float_t	12	Location of the destination coordinate system origin in source coordinates. This column vector is denoted by $\boldsymbol{r}_{0d}$ in the text.
		104	Total size.

Table A.9 The coordinate transformation structure *coord_trans_t*.

The meaning of the transformation members (*rot*, *move*, *invrot*, and *invmove* is specified as follows. Let

$$\boldsymbol{r}_s = \begin{bmatrix} x_s & y_s & z_s \end{bmatrix}^T \text{ and } \boldsymbol{r}_d = \begin{bmatrix} x_d & y_d & z_d \end{bmatrix}^T$$

be the location vectors in source and destination coordinate systems, respectively. In the above, superscript T denotes the matrix transpose). Using the definitions from Table B.8 the coordinate transformations between the two coordinate frames are then

$$\boldsymbol{r}_d = \boldsymbol{R}\boldsymbol{r}_s + \boldsymbol{r}_{0s}$$

and

$$\boldsymbol{r}_s = \boldsymbol{R}^T \boldsymbol{r}_d + \boldsymbol{r}_{0d} .$$

If (unit) orientation vectors like surface normals are transformed, the second term has to be omitted from the above equations.

Especially in computer graphics it is customary to combine $\boldsymbol{R}$ and $\boldsymbol{r}_{0s}$. into a single 4×4 matrix and to consider aumented row vectors instead of column vectors. Setting

$$\boldsymbol{v}_s = \begin{bmatrix} \boldsymbol{r}_s^T & 1 \end{bmatrix} \text{ and } \boldsymbol{v}_d = \begin{bmatrix} \boldsymbol{r}_d^T & 1 \end{bmatrix}$$

we find

$$\boldsymbol{v}_d = \boldsymbol{v}_s \boldsymbol{T}_{sd} ,$$

where we have introduced the 4×4 transformation matrix

$$\boldsymbol{T} = \begin{bmatrix} \boldsymbol{R}^T & \boldsymbol{0} \\ \boldsymbol{r}_{0s}^T & 1 \end{bmatrix} .$$

In this notation, the transformation of orientations is particularly simple: the fourth component in the augmented vector is replaced by zero and the transformation matrix remains intact.

A.5 Multidimensional data

It is often necessary to encode a vector or a matrix of data of a given data type. This section discusses encoding of vectors and matrices.

A.5.1 Vector

There are two possibilities for encoding a vector:

1. The *type* member of the tag is set to the base type as for scalar values. The *size* member indicates how many items of the given type are present, i.e. *size = number_of_items * sizeof(type)*. This is the most usual encoding, see Section A.2.
2. The vector is encoded as a one-dimensional matrix. This encoding scheme has not been used so far.

A.5.2 Matrix representation

As described in Section A.2 the *type* member of a tag stores the information on the basic data type in the first 12 bits. The rest of the bits are used for storing information on whether the data is in matrix format, and which matrix format is employed. The matrix information can be stored as either a dense matrix or a sparse matrix.

Table A.10 describes the mask bits that can be used ford requesting information on the matrix representation. The information can be requested applying values in Table A.11 and Table A.12 using bitwise or operator on the type member of the tag.

Mask name	Mask bits	Description
base_mask	0x00000FFF	The bits that store the basic data type of the data.
fs_mask	0xFF000000	The bits that store information on whether the data is scalar or matrix.
mc_mask	0x00FF0000	The bits the store compession type of the matrix.

Table A.10 The mask to get information on the matrix type.

Name	Bits	Description
fs_scalar	0x00000000	The data is in scalar format (single scalar or a linear array).
fs_matrix	0x40000000	The data is in matrix format.

Table A.11 The possible bits to store for fs_mask bits.

Name	Mask bits	Description
mc_dense	0x00000000	The matrix is stored in dense representation (see Section A.5.2.1).
mc_ccs	0x00100000	The matrix is stored in column-compressed sparse representation (see Section A.5.2.2).
mc_rcs	0x00200000	The matrix is stored in row-compressed sparse representation (see Section A.5.2.2).

Table A.12 The possible bits to store for mc_mask bits.

A.5.2.1 Dense matrix

In a dense matrix all elements are defined. The length of the data is then:

<*num_elements*>*sizeof(<*datatype*>)+ (<*num_dim*>+1)*sizeof(*int32_t*)

where <*num_elements*> is the number of matrix elements, and <*num_dim*> is the number of dimensions in the matrix. The last four bytes of the tag data indicate the number of matrix dimensions. The <*num_dim*>*4 bytes before the matrix dimensionality information indicate the size of the matrix in each dimension starting from the dimension that runs fastest in memory (see below).

At present only up to nine-dimensional matrices are allowed. This is an arbitrary restriction set to ease the implementation of routines accessing the matrices.

Consider for example, a 20 by 30 two-dimensional matrix containing *float_t* data. The matrix data then consists of the following:

- 600 floats for the matrix data,
- integer 30 to indicate the number of columns,
- integer 20 to indicate the number of rows, and
- integer 2 to indicate a two-dimensional matrix.

Note: The matrix is stored in row major as in C language (as opposed to, for example, Fortran). For example, 2-dimensional matrix each row is laid in file after each other. In C, you can easily set up an array of float pointers to access the matrix data using double indexing in the following way:

```
m = malloc(nx*sizeof(float *));
for (k = 0; k < nx; k++)
  m[k] = data+k*ny;
```

The matrix can be now addressed with double indexing (`m[j][k]`) and the following code frees the matrix

```
free(m[0]);
free(m);
```

A.5.2.2 Sparse matrix

In a sparse matrix, most of the elements have a zero-value. To save diskspace the zeroes are omitted, and a sparse matrix can be stored in row-compressed (*mc_rcs*) or column-compressed (*mc_ccs*) packing. See Table A.12 for information on the matrix representation bits.

The non zero data elements (the number is <*num_elements*>) is stored first as a linear array. Then there are <*num_elements*> row (column com-

pressed) or column (row compressed) indices as an array. The next <*size_of_dimension*> bytes store the column (column compressed) or row (row compressed) start indices as a linear array, where <*size_of_dimension*> is the column or row dimension, respectively. The last four bytes of the tag data indicate the number of matrix dimensions. The <num_dim>*2*4 bytes before the matrix dimensionality information indicate the number of non zero elements and the size of the matrix in each dimension starting from the most significant dimension (see below for an example).

At present only up to nine-dimensional matrices are allowed. This is an arbitrary restriction set to ease the implementation of routines accessing the matrices.

Consider for example, a 306 by 306 two-dimensional matrix containing *double_t* data where only 5360 values are non-zero. The column- (or row-) compressed sparse matrix data then consists of the following:

- 5360 doubles for the matrix data,
- row index array (5360 ints),
- column start index array (306 ints),
- integer 5360 to indicate the number of non-zero elements,
- integer 5360 (repeated for backward compatibility),
- integer 306 to indicate the number of rows,
- integer 306 to indicate the number of columns, and
- integer 2 to indicate a two-dimensional matrix.

Row-order compression is similar to column compression except that dimensions 1 and 2 are interchanged. Structurally it is exactly same as the column-order compression of the transpose.

B

APPENDIX B Enumerated types directory

The id values (Id column in section below) are encoded using *int32_t*.

B.1 Value units

Table B.1 describes the `enum(units)`, which gives the different units that can be used for values in FIFF.

Name	Id	Description
none	-1	<No unit>
unitless	0	unitless
m	1	meter
kg	2	kilogram
sec	3	second
A	4	ampere
K	5	Kelvin
mol	6	mole
rad	7	radian
sr	8	steradian
cd	9	candela
mol_m3	10	mol/m^3
Hz	101	herz
N	102	Newton
Pa	103	pascal
J	104	joule
W	105	watt

Table B.1

Name	Id	Description
C	106	coulomb
V	107	volt
F	108	farad
Ohm	109	ohm
Mho	110	one per ohm
Wb	111	weber
T	112	tesla
H	113	Henry
Cel	114	celcius
lm	115	lumen
lx	116	lux
V_m2	117	V/m^2
T_m	201	T/m
Am	202	Am
Am_m2	203	Am/m^2
Am_m3	204	Am/m^3

Table B.1

B.2 Value multipliers

Table B.2 describes `enum(unitm)`, which gives the different multipliers applicable to values in FIFF.

Name	Id	Description
e	18	10^18
pet	15	10^15
t	12	10^12
gig	9	10^9
meg	6	10^6
k	3	10^3

Table B.2

Name	Id	Description
h	2	10^2
da	1	10^1
none	0	10^0
d	-1	10^-1
c	-2	10^-2
m	-3	10^-3
mu	-6	10^-6
n	-9	10^-9
p	-12	10^-12
f	-15	10^-15
a	-18	10^-18

Table B.2

B.3 Channel types

Table B.3 describes `enum(ch_type)`, which gives all the supported channel types.

Name	Id	Description
magn	1	MEG channel (magnetic channel)
el	2	EEG channel (electric channel)
stim	3	Stimulus channel
bio	102	BIO channel
mcg	201	MCG channel
eog	202	EOG channel
magn_ref	301	MEG reference channel (CTF type)
emg	302	EMG channel
ecg	402	ECG channel
misc	502	Misc channel
resp	602	Respiration monitoring

Table B.3

Name	Id	Description
quat0	700	Quaternion channel q0
quat1	701	Quaternion channel q1
quat2	702	Quaternion channel q2
quat3	703	Quaternion channel q3
quat4	704	Quaternion channel q4
quat5	705	Quaternion channel q5
quat6	706	Quaternion channel q6
hpi_goodness	707	HPI fit goodness
hpi_error	708	HPI fit error
hpi_movement	709	HPI coil movement
seeg	802	Stereo EEG channel
syst	900	System state channel (for internal use)
ecog	902	Electrocorticogram channel
ias	910	Internal active shielding channel
exci	920	Excitation channel
dipole_wave	1000	Dipole time curve
goodness_fit	1001	Goodness of fit
fnirs	1100	Functional near-infrared spectroscopy

Table B.3

B.4 Coil types

Table B.4 describes `enum(coil)`, which gives all the different supported coil types for magnetic channels.

Name	Id	Description
none	0	The location info contains no data
eeg	1	EEG electrode position in r0
nm_122	2	Neuromag 122 coils
nm_24	3	Old 24 channel system in HUT

Table B.4

Name	Id	Description
nm_mcg_axial	4	The axial devices in the HUCS MCG system
eeg_bipolar	5	Bipolar EEG electrode position
eeg_csd	6	CSD-transformed EEG electrode
dipole	200	Time-varying dipole definition. The coil info contains dipole location (r0) and direction (ex)
fnirs_hbo	300	FNIRS oxyhemoglobin
fnirs_hbr	301	FNIRS deoxyhemoglobin
fnirs_raw	302	fNIRS raw
fnirs_od	303	fNIRS optical density
mcg_42	1000	For testing the MCG software
point_magnetometer	2000	Simple point magnetometer
axial_grad_5cm	2001	Generic axial gradiometer
vv_planar_w	3011	VV prototype wirewound planar sensor
vv_planar_t1	3012	Vectorview SQ20483N planar gradiometer
vv_planar_t2	3013	Vectorview SQ20483N-A planar gradiometer
vv_planar_t3	3014	Vectorview SQ20950N planar gradiometer
vv_planar_t4	3015	MEG-MRI proto system planar gradiometer
vv_mag_w	3021	VV prototype wirewound magnetometer
vv_mag_t1	3022	Vectorview SQ20483N magnetometer
vv_mag_t2	3023	Vectorview SQ20483-A magnetometer
vv_mag_t3	3024	Vectorview SQ20950N magnetometer
vv_mag_t4	3025	MEG-MRI proto system magnetometer
magnes_mag	4001	Magnes WH magnetometer
magnes_grad	4002	Magnes WH gradiometer
magnes_ref_mag	4003	Magnes WH reference magnetometer
magnes_ref_gdia	4004	Magnes WH diagonal reference gradiometer

Table B.4

Name	Id	Description
magnes_ref_goff	4005	Magnes WH offdiagonal reference gradiometer
ctf_grad	5001	CTF axial gradiometer
ctf_ref_mag	5002	CTF reference magnetometer
ctf_ref_grad	5003	CTF reference gradiometer
ctf_ref_goff	5004	CTF offdiagonal reference gradiometer
kit_grad	6001	KIT gradiometer
kit_ref_mag	6002	KIT reference magnetometer
baby_grad	7001	Baby-SQUID gradiometer
baby_mag_inner	7002	BabyMEG inner layer magnetometer
baby_mag_outer	7003	BabyMEG outer layer magnetometer
baby_mag_ref	7004	BabyMEG reference layer magnetometer
artemis123_grad	7501	Artemis123 gradiometer
artemis123_ref_mag	7502	Artemis123 reference magnetometer
artemis123_ref_grad	7503	Artemis123 reference gradiometer
quspin_zfopm_mag	8001	QuSpin ZFOPM magnetometer Gen 1
quspin_zfopm_mag2	8002	QuSpin ZFOPM magnetometer Gen 2
kriss_grad	9001	KRISS gradiometer
compumedics_adult_grad	9101	Compumedics adult gradiometer
compumedics_pediatric_grad	9102	Compumedics pediatric gradiometer

Table B.4

B.5 Acquisition system type

Table B.5 describes `enum(dacq_system)`, which gives the different acquisition system types.

Name	Id	Description
dau	0	HP3582 DAU based Neuromag122 system.
vxi	1	VXI based Neuromag122 or mcg system.

Table B.5

Name	Id	Description
rpu	2	RPU based Neuromag122 or Vectorview system.
orion	3	Elekta Neuromag(R).
triux	4	TRIUX(TM) or TRIUX(TM) neo

Table B.5

B.6 Gantry type

Table B.6 describes `enum(gantry_type)`, which gives the different types of the gantry.

Name	Id	Description
fixed	0	Fixed gantry
uni_axial	1	Uni-axial, rotation axis parallel to device y-axis
free	2	Freely adjustable 5 degrees of freedom dewar

Table B.6

B.7 Different aspects of data

Table B.7 describes `enum(aspect)`, which gives the different aspects of data, used to describe evoked data type.

Name	Id	Description
average	100	Normal average of epochs
std_err	101	Std. error of mean
single	102	Single epoch cut out from the continuous data
subaverage	103	Partial average (subaverage)
altaverage	104	Alternating subaverage
sample	105	A sample cut out by graph

Table B.7

Name	Id	Description
power_density	106	Power density spectrum
dipole_wave	200	Dipole amplitude curve
ifii_low	1100	Lower bound of SQUID modulation (in I-PHI curves)
ifii_high	1101	Higher bound of SQUID modulation (in I-PHI curves)
gate	1102	Gate setting of SQUID

Table B.7

B.8 Coordinate system definition

Table B.8 describes `enum(coord)`, which gives different possible coordinate *systems.

Name	Id	Description
unknown	0	Unknown coordinate system
device	1	MEG device coordinate system
isotrak	2	3D Digitizer coordinate system
hpi	3	Position indicator system coordinates
head	4	Head coordinates, MEGIN convention
data_volume	5	Coordinate system of the data volume
data_slice	6	Coordinate system of a slice (e.g. MRI image)
data_display	7	Coordinate system of the data display
dicom_device	8	Coordinate system of the imaging device, DICOM convention
imaging_device	9	Coordinate system of the structural imaging device
voxel_data	10	Coordinate system of multidimensional voxel values
atlas_head	11	Head coordinates of the atlas std head
torso	100	Torso coordinates

Table B.8

Name	Id	Description
tufts_eeg	300	Tufts EEG coordinate system
ctf_device	1001	CTF device coordinate system
ctf_head	1004	CTF head coordinate system
mri_voxel	2001	MRI voxel coordinates
ras	2002	Surface RAS coordinates with non-zero origin
mni_tal	2003	MNI Talairach coordinates
fs_tal_gtz	2004	FreeSurfer Talairach (MNI z > 0) coordinates
fs_tal_ltz	2005	FreeSurfer Talairach (MNI z < 0) coordinates
fs_tal	2006	FreeSurfer Talairach coordinates

Table B.8

B.9 Point definitions

Table B.9 describes `enum(point)`, which gives the different supported point types, e.g. used in digitization.

Name	Id	Description
cardinal	1	Cardinal point
hpi	2	Head position indicator (HPI) coil
eeg	3	EEG electrode
ecg	3	ECG electrode
extra	4	Extra point
head_surface	5	Point on the surface of the head

Table B.9

B.10 Cardinal points for brain

Table B.10 describes `enum(cardinal_point)`, which gives the cardinal points for brain imaging.

Name	Id	Description
lpa	1	Left pre-auricular
nasion	2	Nasion
rpa	3	Right pre-auricular
inion	4	Inion

Table B.10

B.11 Cardinal points for heart

Table B.11 describes `enum(cardinal_point_cardiac)`, which gives the cardinal points for cardiac imaging.

Name	Id	Description
chest_left	1	Left chest
sternum	2	Sternum
chest_right	3	Right chest

Table B.11

B.12 SSS job options

Table B.12 describes `enum(sss_job)`, which gives the sss tasks that can be applied to data.

Name	Id	Description
sss_job_nothing	0	No SSS, just copy input to output
sss_job_ctc	1	No SSS, only cross-talk correction
sss_job_filter	2	Spatial maxwell filtering
sss_job_virt	3	Transform data to another sensor array
sss_job_head_pos	4	Estimate head positions, no SSS

Table B.12

Name	Id	Description
sss_job_movec_fit	5	Estimate and compensate head movement
sss_job_movec_qua	6	Compensate head movement from previously estimated head positions
sss_job_rec_all	7	Reconstruct inside and outside signals
sss_job_rec_in	8	Reconstruct inside signals
sss_job_rec_out	9	Reconstruct outside signals
sss_job_st	10	Spatio-temporal maxwell filtering
sss_job_tproj	11	Temporal projection, no SSS
sss_job_xsss	12	Cross-validation SSS
sss_job_xsub	13	Cross-validation subtraction, no SSS
sss_job_xwav	14	Cross-validation noise waveforms
sss_job_ncov	15	Noise covariance estimation
sss_job_scov	16	SSS sample covariance estimation

Table B.12

B.13 Projection (SSP) types

Table B.13 describes `enum(proj_item)`, the projection items that can be represented.

Name	Id	Description
none	0	Not known.
field	1	Magnetic field shape.
dip_fix	2	Fixed (in head coordinates) position dipole
dip_rot	3	Rotating (in head coordinates) dipole
homog_grad	4	Homogeneous gradient
homog_field	5	Homogeneous fiels
eeg_avref	10	Linear projection related to EEG average reference

Table B.13

B.14 Method by which projection is defined

Table B.14 describes `enum(proj_by)`, the method by which projection is defined.

Name	Id	Description
complement	0	Defined by the basis of the complement space
space	1	Defined by the basis of the space itself

Table B.14

B.15 Volumetric image data types

Table B.15 describes `enum(volume_type)`, which gives the different supported volumetric image data types.

Name	Id	Description
unknown	0	Not known
mri	1	Normal structural MRI scan
angio	2	Enhanced Angio MRI scan
spectroscopy	3	MRI spectroscopy scan
bold	4	Functional MRI image
diffusion	5	Diffusion tensor volume
ct	6	Computed tomography scan
segment_mask	7	Segmentation mask
distance	8	Voxel distance map
transform	9	Transformation vectors
current_stat	10	Current density statistical measure, strength, density, t_stat etc.
conductivity	11	Conductivity

Table B.15

B.16 Format of volume data files

Table B.16 describes `enum(mri_format)`, which gives the different file formats for volumetric image data.

Name	Id	Description
unknown	0	Unknown
magnetom_short	1	Magnetom short
magnetom_long	2	Magnetom long
merit	3	Merit
signa	4	Signa
pixel_raw	5	Raw pixels
pixel_raw_packed	6	Raw pixels packed
magnetom_new	7	Magnetom new
fiff	8	FIFF
acr_nema	9	ACR NEMA
dicom_3	10	DICOM 3
vista	11	Vista

Table B.16

B.17 Pixel encoding of volume data

Table B.17 describes `enum(mri_pixel)`, which gives the possible pixel types of volume data.

Name	Id	Description
unknown	0	Unknown encoding
byte	1	The pixels are unsigned bytes.
word	2	The pixels are unsigned shorts (2 bytes) in big-endian byte order.
swap_word	3	The pixels are unsigned shorts (2 bytes) in little-endian byte order.
float	4	The pixels are 4-byte IEEE floats.

Table B.17

Name	Id	Description
byte_indexed_color	5	The pixels are indexes to a 8-bit color map.
byte_rgb_color	6	The pixels are represented by three byte values indicating the red, green, and blue intensities (0..255).
byte_rle_rgb_color	7	RGB values with run-length enconding.
bit_rle	8	A run-length encoded bitmap. This will eventually be used for representation of segmentation regions.
signed_word	9	The pixels are signed shorts (2 bytes) in big-endian byte order.
signed_swap_word	10	The pixels are signed shorts (2 bytes) in little-endian byte order.

Table B.17

B.18 Diffusion parameter maps

Table B.18 describes `enum(diffusion_param)`, which gives different diffusion parameters mapped in volumetric images.

Name	Id	Description
fa	1	Fractional anisotropy
adc	2	Apparent diffusion coefficient
linear	3	Linear measure
planar	4	Planar measure
spherical	5	Spherical measure
eigen_max	6	Maximum eigen value
comp_3	7	First three eigenvalues
comp_3_fa	8	First three eigenvalues scaled with fa
tensor_6	9	Tensor value
other	15	Other parameter

Table B.18

B.19 Conductor model types

Table B.19 describes `enum(cond_model)`, which gives the different supported conductor models.

Name	Id	Description
unknown	0	Not known
sphere	1	Spherically symmetric
bem_homog	2	Homogeneous BEM model (single layer)
bem	3	Multilayer BEM model

Table B.19

B.20 The BEM approximation method

Table B.20 describes `enum(bem_approx)`, which gives the different approximation methods used in boundary element modelling.

Name	Id	Description
const	1	The constant potential approach
linear	2	The linear potential approach

Table B.20

B.21 Surface identifiers

Table B.21 describes `enum(bem_surf_id)`, the surfaces that can be associated with anatomical structures.

Name	Id	Description
unknown2	-1	Undefined
unknown	0	Not known
brain	1	Brain surface
csf	2	Cerospinal fluid
skull	3	Skull bone
head	4	Scalp surface

Table B.21

Name	Id	Description
blood	11	Blood mass
heart	12	Heart surface
lungs	13	Lung surface
torso	14	Thorax surface
nm122	21	NM122 sensor array
unit_sphere	22	Unit sphere
vv	23	VectorView sensor array

Table B.21

B.22 Role

These are the values for `enum(ref_role)`, which gives the role of a reference.

Name	Id	Description
prev_file	1	Previous file
next_file	2	Next file

Table 2.22

B.23 Hand

These are the values for `enum(hand)`, which gives the handedness of a subject..

Name	Id	Description
right	1	Righthanded
left	2	Lefthanded

Table 2.23

B.24 Map surface

These are the values for `enum(map_surf)`, which gives the supported contour surface types..

Name	Id	Description
sensors	0	Sensor array surface
head	1	Head surface
sphere	2	Spherical surface

Table 2.24

B.25 Next

These are the values for `enum(next)`, which gives special values for the next tag member..

Name	Id	Description
none	-1	None
seq	0	Sequence

Table 2.25

B.26 Sex

These are the values for `enum(sex)`, which gives the sex of a subject..

Name	Id	Description
unknown	0	Unknown gender
male	1	Male
female	2	Female

Table 2.26

C

APPENDIX C Tag directory

This chapter contains a directory of all presently defined tags. Tags are described in groupings which collect the tags according to the context in which they are used. The numeric values of the constants are also listed. A brief explanation of the the tag is included as well as the data types that may be used to encode the data. The data types are described in Appendix A.

Vectors and matrices are described with the following syntax:

Format	Description
tag*	Implicit vector, size undefined or defined by the context
tag*(3)	Implicit vector, 3 components
tag[*,*]	Matrix, size or defined by context
tag[n,m]	Matrix of size n times m

Table 3.1

C.1 Acquisition control

These tags never occur in FIFF files stored on disk. They are only used in the communication from the real-time acquisition system to the acquisition workstation. The program *dacq_server_peer*, which receives the data from the real-time system distributes these control tags to all its clients. It is up to the clients to perform the necessary actions required when these tags are received.

Symbolic name	Tag id	Data type	Unit	Description
new_file	1	void	-	New file should be started.
close_file	2	void	-	Done, close the file being saved.

Table 3.2

Symbolic name	Tag id	Data type	Unit	Description
discard_file	3	void	-	Done, discard the file being saved.
error_message	4	string	-	Acquisition error notifier.
suspend_reading	5	void	-	Transmission stops temporarily.
fatal_error_message	6	string	-	Fatal acquisition error notifier.
connection_check	7	void	-	Dummy object for testing transmission.
suspend_filing	8	void	-	Stop filing data command.
resume_filing	9	void	-	Start filing data command.
raw_prebase	10	int32	card	Prebase length (in buffers) change request.
raw_pick_list	11	int32*	-	Channel order in data.
echo	12	void	-	Channel probe, response expected.
resume_reading	13	void	-	Suspended transmission continues.
dacq_system_type	14	enum(dacq_system)	-	Type of the acquitition system.
select_raw_ch	15	string	-	Instruct rawdisp to select this channel.
playback_mode	16	void	-	Data are being played back from the hard disks.
continue_file	17	string	-	Used to inform that data is saved into a continuation file.
jitter_max	18	float	s	Maximum jitter in data communication blocks (seconds)
stream_segment	19	int32	-	Data stream segment

Table 3.2

C.2 Common tags

The tags listed below are common to all FIFF data files.

Symbolic name	Tag id	Data type	Unit	Description
file_id	100	id1.1	-	Id of the file
dir_pointer	101	int32	-	Pointer to directory
dir	102	int32	-	Tag directory
block_id	103	id1.1	-	Id of a block
block_start	104	int32	enum	Start of a composite object (block)
block_end	105	int32	enum	End if composite object (block)
free_list	106	int32	ord	Pointer to fre block list.
free_block	107	void	-	Unused space
nop	108	void	-	Reserved space
parent_file_id	109	id1.1	-	Id of the file from which this was generated
parent_block_id	110	id1.1	-	Id of a block from which this data is derived from
block_name	111	string	-	Name of a block
block_version	112	string	-	Version number of the block format of this block
creator	113	string	-	Program that created the file (string)
modifier	114	string	-	Program that modified the file (string)
ref_role	115	enum(role)	-	What is the role of this reference
ref_file_id	116	id1.1	-	File id of the referenced object
ref_file_num	117	int32	ord	File number of the referenced file
ref_file_name	118	string	-	Name hint for the refereced file
ref_block_id	120	id1.1	-	Id of a referenced block

Table 3.3

C.3 Common data tags

Table below lists tags common to MEG data files.

Symbolic name	Tag id	Data type	Unit	Description
dacq_pars	150	string	-	Megacq saves the parameters in these
dacq_stim	151	string	-	Megacq saves stimulus parameters in these
device_type	152	string	-	Device type from NM_DEVICE
device_model	153	string	-	Device model from NM_DEVICE_MODEL
device_serial	154	string	-	Device serial from NM_SERIAL_NUMBER
device_site	155	string	-	Device site from NM_SITE
he_level_raw	156	float	-	Helium level % before position correction
helium_level	157	float	-	Helium level % after position correction
orig_file_guid	158	string	-	Original file GUID
utc_offset	159	string	-	UTC offset of related meas_date (sHH:MM)
nchan	200	int32	card	Number of channels
sfreq	201	float	Hz	Sampling frequency (Hz)
data_pack	202	int32	ord	How the raw data is packed
ch_info	203	ch_info_rec	-	Channel descriptor
meas_date	204	int32	uxtime	Measurement date (in Unix time encoding)
subject	205	string	-	<obsolete>
description	206	string	-	(Textual) Description of an object
nave	207	int32	card	Number of averages
first_sample	208	int32	ord	The first sample of an epoch
last_sample	209	int32	ord	The last sample of an epoch
aspect_kind	210	enum(aspect)	-	Aspect label

Table 3.4

Symbolic name	Tag id	Data type	Unit	Description
ref_event	211	int32	ord	Reference event
experimenter	212	string	-	Experimenter name
dig_point	213	dig_point	m	Digitization point
ch_pos_vec	214	ch_pos_rec	-	Channel positions (obsolete)
hpi_slopes	215	float*	T,T/m	HPI data
hpi_ncoil	216	int32	card	Number of HPI coils
req_event	217	int32	ord	Required event
req_limit	218	float	s	Window for required event
lowpass	219	float	Hz	Analog lowpass
bad_chs	220	int32*	ord	List of bad channels
artef_removal	221	int32	ord	Artefact removal
coord_trans	222	coor_trans_rec	-	Coordinate transformation
highpass	223	float	Hz	Analog highpass
ch_cals_vec	224	float*	rel	Channel calibration (obsolete)
hpi_bad_chs	225	int32*	ord	List of channels considered to be bad in hpi
hpi_corr_coeff	226	float*	rel	Hpi curve fit correlations
event_comment	227	string	-	Comment about the events used in averaging
no_samples	228	int32	card	Number of samples in an epoch
first_time	229	float	s	Time scale minimum
subave_size	230	int32	card	Size of a subaverage
subave_first	231	int32	ord	The first epoch
name	233	string	-	Intended to be a short name
dig_string	234	dig_string	-	String of digitized points
line_freq	235	float	Hz	Basic line interference frequency
hpi_coil_freq	236	float	Hz	HPI coil excitation frequency
signal_channel	237	string	-	Signal channel name

Table 3.4

C.4 Head position indicator (HPI) data

This section describes the tags used to store head position indicator (HPI) measurement and fitting data.

Symbolic name	Tag id	Data type	Unit	Description
hpi_coil_moments	240	float	T/m	Estimated moment vectors for the HPI coil magnetic dipoles
hpi_fit_goodness	241	float	%	Three floats indicating the goodness of fit
hpi_fit_accept	242	bit-mask(hpi_accept)	-	Bitmask indicating acceptance (see enum hpi_accept)
hpi_fit_good_limit	243	float	%	Limit for the goodness-of-fit
hpi_fit_dist_limit	244	float	m	Limit for the coil distance difference
hpi_coil_no	245	int32	ord	Coil number listed by HPI measurement
hpi_coils_used	246	int32*	ord	List of coils finally used when the transformation was computed
hpi_digitization_order	247	int32*	ord	Which Isotrak digitization point corresponds to each of the coils energized

Table 3.5

C.5 Channel info tags

This section lists the tags used for information of channels.

Symbolic name	Tag id	Data type	Unit	Description
ch_scan_no	250	int32	ord	Channel scan number. Corresponds to fiffChInfoRec.scanNo field
ch_logical_no	251	int32	ord	Channel logical number. Corresponds to fiffChInfoRec.logNo field
ch_kind	252	enum(ch_type)	-	Channel type. Corresponds to fiffChInfoRec.kind field

Table 3.6

Symbolic name	Tag id	Data type	Unit	Description
ch_range	253	float	rel	Conversion from recorded number to (possibly virtual) voltage at the output
ch_cal	254	float	rel	Calibration coefficient from output voltage to some real units
ch_pos	255	ch_pos_rec	-	Channel position
ch_unit	256	enum(unit)	-	Unit of the data
ch_unit_mul	257	int32	rel	Unit multiplier exponent.
ch_dacq_name	258	string	-	Name of the channel in the data acquisition system. Corresponds to fiffChInfoRec.name.

Table 3.6

C.6 Signal space separation (SSS)

These tags store the information about the signal space separation (SSS) parameters and data.

Symbolic name	Tag id	Data type	Unit	Description
sss_frame	263	int32	-	SSS expansion coordinate frame
sss_job	264	enum(sss_job)	-	SSS job selection
sss_origin	265	float*(3)	m	SSS expansion origin
sss_ord_in	266	int32	-	SSS inside expansion order
sss_ord_out	267	int32	-	SSS outside expansion order
sss_nmag	268	int32	-	SSS number of MEG channels
sss_components	269	int32*	-	SSS expansion component selections
sss_cal_chans	270	int32[*,*]	-	SSS calibration ch nrs and types
sss_cal_corrs	271	float[*,*]	-	SSS calibration adjustments
sss_st_corr	272	float	-	SSS ST subspace correlation
sss_base_in	273	double[*,*]	-	SSS inside basis matrix
sss_base_out	274	double[*,*]	-	SSS outside basis matrix

Table 3.7

Symbolic name	Tag id	Data type	Unit	Description
sss_base_virt	275	double[*,*]	-	SSS virtual basis matrix
sss_norm	276	double	-	SSS froebious norm of inside basis
sss_iterate	277	int32	-	SSS nr of iterations
sss_nfree	278	int32	-	SSS number of degrees of freedom
sss_st_length	279	float	-	SSS ST buffer length

Table 3.7

C.7 Gantry information

This section section describes tags for storing information on properties of the gantry.

Symbolic name	Tag id	Data type	Unit	Description
gantry_type	280	enum(gantry_type)	-	The type of the gantry
gantry_model	281	string	-	The modal of the gantry
gantry_angle	282	int	-	Tilt angle of the dewar in degrees

Table 3.8

C.8 Signal data bits

These tags describe the signal data bits.

Symbolic name	Tag id	Data type	Unit	Description
data_buffer	300	int16|int32|float	-	Buffer containing measurement data
data_skip	301	int32	card	Data skip in buffers
epoch	302	float[*,*]|oldpack	-	Buffer containing one epoch and channel
data_skip_samp	303	int32	card	Data skip in samples

Table 3.9

Symbolic name	Tag id	Data type	Unit	Description
data_buffer2	304	databuffer2	-	Data buffer with int32 time channel
time_stamp	305	int32*(3)	-	Measurement time stamp, first element seconds, second microseconds, and third sample number

Table 3.9

C.9 Patient information

These tags store the information about the patient or subject the data file refers to. Patient information is further discussed in Section 3.4.4.

Symbolic name	Tag id	Data type	Unit	Description
subj_id	400	int32	ord	Subject ID
subj_first_name	401	string	-	First name of the subject
subj_middle_name	402	string	-	Middle name of the subject
subj_last_name	403	string	-	Last name of the subject
subj_birth_day	404	julian	-	Birthday of the subject
subj_sex	405	enum(sex)	-	Sex of the subject
subj_hand	406	enum(hand)	-	Handedness of the subject
subj_weight	407	float	kg	Weight of the subject
subj_height	408	float	m	Height of the subject
subj_comment	409	string	-	Comment about the subject
subj_his_id	410	string	-	ID used in the Hospital Information System

Table 3.10

C.10 Project information

It is possible to group the acquired data according to projects. These tags store the information about the project under which the data file has been acquired..

Symbolic name	Tag id	Data type	Unit	Description
proj_id	500	int32	-	Project ID
proj_name	501	string	-	Project name
proj_aim	502	string	-	Project description
proj_persons	503	string	-	Persons participating in the project
proj_comment	504	string	-	Comment about the project

Table 3.11

C.11 Event lists

These tags can be used for storing event list information.

Symbolic name	Tag id	Data type	Unit	Description
event_channels	600	int32*	ord	Event channel numbers
event_list	601	int32*(n*3)	-	List of events, 3 integers per event: [number of samples, before, after]
event_channel	602	string	-	Event channel name
event_bits	603	int32*(4)	-	Event bits array describing transition, 4 integers: [from_mask, from_state, to_mask, to_state

Table 3.12

C.12 SQUID characteristics

These tags are used for storing SQUID characteristics etc.

Symbolic name	Tag id	Data type	Unit	Description
squid_bias	701	int32	arb	Bias setting of a squid
squid_offset	702	int32	arb	Offset setting of a squid
squid_gate	703	int32	arb	Gate setting of a squid

Table 3.13

C.13 Volumetric image data

This section describes the tags related to storing of volumetic image data in FIFF files.

Symbolic name	Tag id	Data type	Unit	Description
volume_type	2001	enum(volume_type)	-	Type of the volume
mri_source_format	2002	enum(mri_format)	-	File format of referenced image data
mri_pixel_encoding	2003	enum(pixel_encoding)	-	Pixel type of the stored data
mri_pixel_data_offset	2004	int32	ord	Offset to the pixel data in the referenced data
mri_pixel_scale	2005	float	rel	Scaling factor from the disk data to unsigned char format
mri_pixel_data	2006	byte*	rel	Pixel data stored in the Fiff file
mri_pixel_overlay_encoding	2007	enum(mri_pixel)	-	Pixel type of the stored overlay data
mri_pixel_overlay_data	2008	byte*	rel	Overlay data stored in the Fiff file
mri_bounding_box	2009	float	m	<obsolete>
mri_width	2010	int32	card	Number of voxels in x direction

Table 3.14

Symbolic name	Tag id	Data type	Unit	Description
mri_width_m	2011	float	m	Size of the volume in x direction
mri_height	2012	int32	card	Number of voxels in y direction
mri_height_m	2013	float	m	Size of the volume in y direction
mri_depth	2014	int32	card	Number of voxels in z direction
mri_depth_m	2015	float	m	Size of the volume in z direction
mri_thickness	2016	float	m	Slice thickness
mri_scene_aim	2017	float*(3)	rel	Direction to which the scene snapshot is taken
mri_calibration_scale	2018	float	rel	Scale from disk values to real world values
mri_calibration_offset	2019	float	rel	Offset for scaling to real world values
mri_orig_source_path	2020	string	-	Path to the source file, where this data is derived from.
mri_orig_source_format	2021	enum(mri_format)	-	Format of the source file.
mri_orig_pixel_encoding	2022	enum(pixel_encoding)	-	Pixel type of the source file
mri_orig_pixel_data_offset	2023	int32	ord	Offset to the pixel data in the source file
mri_time	2024	float	s	Time point of the current volume
mri_voxel_data	2030	variable	-	Volumetric image data
mri_voxel_encoding	2031	enum(pixel_encoding)	-	Encoding for voxel data
voxel_nchannels	2032	int32	ord	Number of channels in a voxel
mri_diffusion_weight	2040	float	s/m^2	Diffusion weigting factor a la LeBihan [s/(m^2)]
mri_diffusion_param	2041	enum(diffusion_param)	-	What diffusion parameter is mapped in the volume
mri_mrilab_setup	2100	<internal>	-	Used internally.

Table 3.14

Symbolic name	Tag id	Data type	Unit	Description
mri_seg_region_id	2200	<internal>	-	Used internally.

Table 3.14

C.14 Conductor models

These tags store information about the conductor model. Sphere model parameters should be grouped into a FIFF_SPHERE block, BEM model parameters into a FIFF_BEM block, and BEM layer parameters into a FIFF_BEM_LAYER bock.

Symbolic name	Tag id	Data type	Unit	Description
conductor_model_kind	3000	enum(cond_model)	-	What kind of conductor model
sphere_origin	3001	float*(3)	m	Origin of sphere model
sphere_coord_frame	3002	enum(coord)	-	Which coordinate frame are we using?
sphere_layers	3003	layer_struct*	-	Array of layer structures
bem_surf_id	3101	enum(bem_surf_id)	-	surface type
bem_surf_name	3102	string	-	surface name
bem_surf_nnode	3103	int32	card	Number of nodes on a surface
bem_surf_ntri	3104	int32	card	Number of triangles on a surface
bem_surf_nodes	3105	float*	card	surface nodes (nnode,3)
bem_surf_triangles	3106	int32*	card	surface triangles (ntri,3)
bem_surf_normals	3107	float*	rel	surface node normal unit vectors (nnode,3)
bem_surf_curvs	3108	float*	1/m	surface node first principal curvature unit vectors (nnode,3)

Table 3.15

Symbolic name	Tag id	Data type	Unit	Description
bem_surf_curv_values	3109	float	1/m	the two curvature values (nnode,2)
bem_pot_solution	3110	float[*,*]	-	The solution matrix
bem_approx	3111	enum(bem_approx)	-	Approximation method for bem computation
bem_coord_frame	3112	enum(coord)	-	The coordinate frame of the model
bem_sigma	3113	float	S/m	Conductivity of a compartment
source_dipole	3201	float*(6)	m,Am	Dipole definition. The dipole is six floats (position and dipole moment)
beamformer_instructions	3300	string	-	Contents of the beamformer instruction file (prototype software)

Table 3.15

C.15 Source modelling software

These tags are used by the source modelling software.

Symbolic name	Tag id	Data type	Unit	Description
xfit_lead_products	3401	double*	-	Lead field matrix data
xfit_map_products	3402	double[*,*]	-	Mapping to isocontour map
xfit_grad_map_products	3403	double[*,*]	-	Mapping to gradient map
xfit_vol_integration	3404	bool	-	Was volume integration used
xfit_integration_radius	3405	float	m	Radius of integration sphere in MNE calculations
xfit_conductor_model_name	3406	string	-	Name of the conductor model used

Table 3.16

Symbolic name	Tag id	Data type	Unit	Description
xfit_conductor_model_trans_name	3407	string	-	Name of the model coordinate transform file
xfit_cont_surf_type	3408	enum(map_surf)	-	Contour surface type in Xfit

Table 3.16

C.16 Signal space projections (SSP)

These tags store the information about the signal space projection (SSP).

Symbolic name	Tag id	Data type	Unit	Description
proj_item_kind	3411	enum(proj_item)	-	Type of the projection defintion. *
proj_item_time	3412	float	s	Time of the field sample (only for proj_item_field)
proj_item_ign_chs	3413	int32*	ord	Channels ignored in the projection definition
proj_item_nvec	3414	int32	card	Number of projection vectors.
proj_item_vectors	3415	float[*,*]	rel	Projection vectors
proj_item_definition	3416	enum(proj_by)	-	How the projection is defined (subspace or its complement)
proj_item_ch_name_list	3417	string	-	Names of the channels of the projection vectors

Table 3.17

C.17 XPlotter

These tags are used by XPlotter software.

Symbolic name	Tag id	Data type	Unit	Description
xplotter_layout	3501	string	-	Xplotter layout tag

Table 3.18

C.18 Cross-talk (CTM)

These tags are used in saving cross-talk matrices..

Symbolic name	Tag id	Data type	Unit	Description
decoupler_matrix	800	sparse	-	Cross-talk correction matrix
ctm_open_amps	801	float[*,*]	-	ctm open-loop amplitudes
ctm_open_phase	802	float[*,*]	-	ctm open-loop phases
ctm_clos_amps	803	float[*,*]	-	ctm closed-loop amplitudes
ctm_clos_phase	804	float[*,*]	-	ctm closed-loop phases
ctm_clos_dote	805	float[*,*]	-	ctm closed-loop dot products
ctm_open_dote	806	float[*,*]	-	ctm open-loop dot products
ctm_exci_freq	807	float	Hz	ctm excitation frequency

Table 3.19

C.19 SSS operator matrices

These tags are used in saving space separation (SSS) operator matrices.

Symbolic name	Tag id	Data type	Unit	Description
sss_operator	290	double[*,*]	-	SSS operator matrix

Table 3.20

Symbolic name	Tag id	Data type	Unit	Description
sss_psinv	291	double[*,*]	-	SSS pseudoinverse matrix
sss_ctc	292	double[*,*]	-	SSS crosstalk and cal correction

Table 3.20

D

APPENDIX D Information entity directory

D.1 Block types

The tags listed in Table D.1 are used to divide a FIFF file into logical blocks and to descriptive and identification information to the blocks. Table D.2 lists the possible aliases for the major block names.

Block name	Id	Description
root	999	Root block of a file (has not been used so far).
meas	100	Measurement
meas_info	101	The information about the acquisition details.
raw_data	102	Raw data
processed_data	103	Processed data
evoked	104	Evoked response data
aspect	105	Used within evoked block to include one aspect of evoked responses like standard average, alternating average, standard error of mean etc.
subject	106	Subject/patient information.
isotrak	107	Head digitization data
hpi_meas	108	HPI measurement
hpi_result	109	Result of a HPI fitting procedure.
hpi_coil	110	Data acquired from one HPI coil.
project	111	Information about the project under which the data were acquired.
continuous_data	112	Continuous data is processed raw data.
void	114	Nothing, reserved space, invalidated data etc.
events	115	Information about the events detected on the trigger channels during data acquisition.

Table D.1 FIFF block types.

Block name	Id	Description
index	116	An index of file id/file name pairs.
dacq_pars	117	The acquisition setup parameters.
ref	118	Reference mechanism.
ias_raw_data	119	Raw data collected with IAS.
ias_aspect	120	Evoked data collected with IAS.
hpi_subsystem	121	Data about HPI state.
phantom_subsystem	122	Data about phantom state.
status_subsystem	123	Data about system status items and coding of related data bits.
device_info	124	Device information.
helium_info	125	Helium information.
channel_info	126	Channel information.
structural_data	200	One or more volume_data.
volume_data	201	Volume data, e.g. MRI/CT set, segmentation etc.
volume_slice	202	Slice of a volume, e.g. MRI/CT image
scenery	203	A set of related (in DICOM terminology ‘secondary capture’) images like the 3D renderings of the brain surface.
scene	204	One image of a scenery.
mri_seg	205	MRI segmentation data
mri_seg_region	206	Description of one segmented region.
sphere	300	A spherically symmetric volume conductor description.
bem	310	A boundary-element model (BEM) description.
bem_surf	311	Describes one BEM surface.
conductor_model	312	One or more conductor model descriptions.
ssp	313	The signal-space projection (SSP) data.
ssp_item	314	One set of vectors of the SSP data.
xfit_aux	315	Auxiliary data created by the source modelling program XFit.
fiff_cov	355	Covariance matrices

Table D.1 FIFF block types.

Block name	Id	Description
bad_channels	359	Bad channel namelist
vol_info	400	Volume info
data_correction	500	Correction was done to data.
channels_decoupler	501	Cross-talk compensation information.
sss_info	502	SSS information.
sss_cal_adjust	503	SSS calibration adjustments.
sss_st_info	504	Information of the SSST parameters.
sss_bases	505	SSS basis matrices.
sss_operator	506	SSS operator matrices
ct_meas	507	Cross-talk measurement.
ias	510	IAS information.
processing_history	900	Processing history. Can contain multiple processing_record's.
processing_record	901	One processing record.

Table D.1 FIFF block types.

Block name	Alias
evoked	mcg_ave
ias_raw_data	smsh_raw_data
ias_aspect	smsh_aspect
structural_data	mri
volume_data	mri_set
volume_slice	mri_slice
scenery	mri_scenery
scene	mri_scene
ssp	xfit_proj
ssp_item	xfit_proj_item
ias	smartshield

Table D.2 Aliases for block types